

08

Electric Vehicle
Codec Technologies India
Internship 2026

Smart Traffic Light with EV Priority

Arduino RFID / GPS priority controller, shared state machine ported to Python and tested in a SimPy intersection

AT A GLANCE

16.8 -> 8.1 s (-52 %)

EV delay at 5 % share, RFID

+4 % (17.0 s)

Non-EV delay penalty at 5 %

24.9 / 50.8

Pre-emptions per hour at 5 / 40 %

60 s, never exceeded

Max non-green time (guard)

1,459 veh/h

Throughput, every configuration

9,130 B flash, 28 %

Firmware compile (recorded)

21 / 21

Controller tests re-run here

5 / 5 (simpy)

Simulation tests (recorded)

VERDICT

TESTS 26/26

21 RE-RUN + 5 RECORDED

SIMULATION ONLY

HARDWARE NOT RUN

FIRMWARE COMPILED

Eswar Mahalingam
Electric Vehicle Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
github.com/eswarmahalingam

1. Objective and brief mapping

Brief: develop a smart traffic system that gives priority to EVs using Arduino with RFID tags or GPS modules, identify EVs approaching intersections and change the light logic dynamically. **Honesty note (README):** no Arduino, RFID reader or GPS module was available, so nothing in this folder has run on physical hardware. The firmware is compile-checked with arduino-cli against the official arduino:avr core (recorded run), the controller logic is ported line-for-line to Python and exercised in a SimPy discrete-event simulation, and a Wokwi diagram lets the controller run in the browser. Every number in results/ comes from the simulation model, not from traffic measurements.

2. Architecture

Arduino Uno with 12 LEDs for a 4-approach intersection (N/E/S/W), two MFRC522 RFID readers on shared SPI, EV tag whitelist in EEPROM with PROGEM fallback, 5 s per-tag debounce, serial commands (n/e/s/w inject, p toggles priority for A/B tests). GPS variant: Nano + NEO-6M + HC-12 vehicle beacon sending \$EV,...*ck messages; the receiver geofences the vehicle, derives the approach from the bearing and the ETA from speed / distance and calls the same state machine. `ev_priority_controller.h` is pure logic with no hardware calls; `simulation/controller.py` is its Python port and a test asserts that the 11 timing constants match.



Fig. 1 - Signal timeline, baseline vs priority: cross-direction greens cut at `GREEN_MIN = 10 s`, the 3 s yellow and 2 s all-red always inserted, same-direction greens stretched to cover the EV arrival.

3. Method and implementation

- State machine: `GREEN_NS 30 s -> YELLOW 3 s -> ALLRED 2 s -> GREEN_EW 30 s ...` Every detection carries an ETA (RFID ETA 8 s for a reader ~100 m upstream at 45 km/h, or the GPS distance / speed). Same-direction EV: extend green to $\max(\text{now} + 15 \text{ s}, \text{arrival} + 5 \text{ s})$, capped at `30 + 20 s`. Cross-direction EV: after `GREEN_MIN = 10 s` and when the cut is due (arrival - 5 s clearance - 3 s lead), end the green; the EV group then gets a green covering the arrival ($\geq 15 \text{ s}$, $\leq 50 \text{ s}$). A green protecting an EV that has not arrived cannot be cut.
- Fairness guard: a group whose previous green was cut cannot be cut again on its next green (deniedRequests). Starvation guard: a group red $\geq 60 \text{ s}$ gets an uncuttable green $\geq 30 \text{ s}$. Requests expire after 30 s. Bound: no group is without green for more than $30 + 20 + 2 \times (3 + 2) = 60 \text{ s}$. `priorityEnabled = false` gives the fixed-time baseline.
- Simulation (traffic_sim.py): 4 approaches, Poisson arrivals 360 veh/h each ($v/c \sim 0.5$ with a 70 s cycle), EV share `p`, point queue with 2 s start-up loss and 2 s headways (1,800 veh/h), controller ticked every 100 ms like `loop()`; 1 h x 10 seeds x {5, 10, 20, 40} % x {baseline, RFID, GPS} = 120 runs in `results/metrics.csv`.

4. Results (results/summary.json, 1 h x 10 seeds, 360 veh/h/approach)

Throughput is 1,459 veh/h in every configuration (nobody left in the queue at the end) and the maximum non-green time is 60 s with priority and 40 s in the baseline in every one of the 120 runs, so priority redistributes delay rather than destroying capacity at this load. At 40 % share the fairness guard denies most cuts (175 denied vs 51 granted per hour) and the benefit shrinks to 12 % - the right behaviour when every third vehicle is 'priority'.

Controller	EV share	EV mean delay	Non-EV mean	Non-EV p95	Worst non-EV wait	EVs stopped	Pre-empt ions/h
Fixed-time baseline	any	16.3-16.8 s	16.3 s	39.6 s	58.7 s	76 %	0
RFID priority (8 s lead)	5 %	8.1 s (-52 %)	17.0 s (+4 %)	43.1 s	94.3 s	63 %	24.9
	10 %	10.7 s (-35 %)	17.7 s (+9 %)	45.2 s	94.5 s	66 %	35.0
	20 %	12.2 s (-25 %)	18.1 s (+11 %)	51.1 s	127.3 s	67 %	48.3
	40 %	14.3 s (-12 %)	19.1 s (+17 %)	54.3 s	107.0 s	68 %	50.8

KEY TAKEAWAY At the 5-10 % EV shares seen on Indian roads today the ETA-aware priority halves EV delay for a 4-9 % penalty on everyone else, with yellow, all-red and a 60 s starvation bound never violated; nothing ran on hardware.

08

Electric Vehicle
Codec Technologies India
Internship 2026

Smart Traffic Light with EV Priority

Arduino RFID / GPS priority controller, shared state machine ported to Python and tested in a SimPy intersection

AT A GLANCE

16.8 -> 8.1 s (-52 %)

EV delay at 5 % share, RFID

+4 % (17.0 s)

Non-EV delay penalty at 5 %

24.9 / 50.8

Pre-emptions per hour at 5 / 40 %

60 s, never exceeded

Max non-green time (guard)

1,459 veh/h

Throughput, every configuration

9,130 B flash, 28 %

Firmware compile (recorded)

21 / 21

Controller tests re-run here

5 / 5 (simpy)

Simulation tests (recorded)

VERDICT

TESTS 26/26

21 RE-RUN + 5 RECORDED

SIMULATION ONLY

HARDWARE NOT RUN

FIRMWARE COMPILED

Eswar Mahalingam
Electric Vehicle Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
github.com/eswarmahalingam

Controller	EV share	EV mean delay	Non-EV mean	Non-EV p95	Worst non-EV wait	EVs stopped	Pre-empt ions/h
GPS priority (20 s lead)	5 %	8.4 s (-50 %)	17.5 s	46.0 s	94.7 s	53 %	14.6
	10 %	10.3 s (-37 %)	19.3 s	54.1 s	144.3 s	55 %	17.8
	20 %	13.1 s (-20 %)	20.3 s	56.6 s	144.2 s	59 %	20.3
	40 %	18.1 s (+11 %)	22.5 s	59.9 s	142.5 s	66 %	14.9

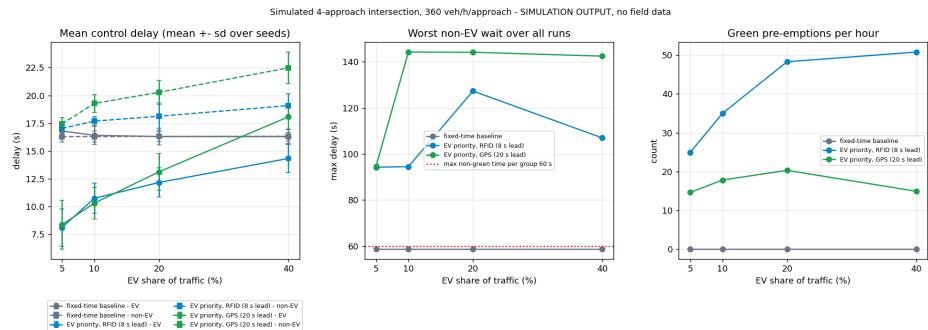


Fig. 2 - Mean delay vs EV share for EVs and non-EVs under baseline, RFID and GPS priority (10 seeds, error bars = SD).

5. Verification

The README records 26 pytest cases passing. Here pytest is not installed and **simpy is unavailable**, so I re-ran the suite with a fixture/parametrize-aware runner in which traffic_sim is stubbed: **21 controller tests passed, 0 failed** in 1.0 s; the **5 simulation-backed tests were not re-run** and their result is the project's recorded run (5 passed). Firmware compilation (arduino-cli) was also not repeated - flash/RAM figures are recorded.

Test	Checks	Result here
test_timing_constants_match_firmware_header, test_gps_variant_uses_identical_header	11 timing constants parsed from the .h equal controller.py; GPS header copy byte-identical	2 PASS
test_never_conflicting_greens[1-5], test_all_red_clearance_always_inserted[1-3]	2 h random EV storms: never two greens, only green->yellow->all-red->green, yellow exactly 3 s, all-red exactly 2 s	8 PASS
test_non_green_time_bounded_by_guards[1-3], test_no_two_consecutive_cuts_on_same_group[1-2]	no group non-green > 60 s; no two consecutive cut greens on a group	5 PASS
test_baseline_is_plain_fixed_time, test_cross_ev_cuts_green_after_min_green_and_clearance, test_gps_eta_cut_is_timed_so_green_shows_before_arrival, test_same_direction_ev_extends_green_up_to_cap, test_request_expires, test_priority_disabled_ignores_detections	70 s fixed cycle with 0 pre-emptions; cut at exactly GREEN_MIN; GPS 20 s ETA cuts at 24 s so green shows 3 s early; extension to 43 s and cap at 50 s; stale request expires; priority off ignores tags	6 PASS
test_ev_delay_lower_with_priority[1-3], test_nonev_max_wait_bounded, test_simulation_conserves_vehicles_and_is_deterministic	SimPy: EV delay with priority < baseline; non-EV max delay <= 3 cycles at 40 %; all vehicles served, deterministic per seed	5 NOT RE-RUN (simpy); recorded PASS

6. Data integrity and declared substitutions

- **No hardware testing at all.** Recorded arduino-cli compile (2 Sept 2026): arduino_ev_priority 9,130 B flash / 549 B RAM with MFRC522 1.4.12; Wokwi build 6,198 B; tag_enroll 6,100 B; GPS beacon 10,578 B; GPS receiver 11,956 B; no warnings from my code. Unknowns: MFRC522 read reliability at speed (a UHF reader is the realistic choice), SPI sharing between two readers, HC-12 packet loss, GPS heading noise at low speed.
- Simulation-only results: point queue with fixed headways, two signal groups, no turns or pedestrians, one mid-load case (360 veh/h/approach), no spill-back past the RFID reader. BOM prices are Sept-2026 estimates +/-30 % (~Rs 1,460 RFID build, ~Rs 1,500 GPS add-on).
- Wokwi: wokwi/diagram.json (Uno + 12 LEDs + 2 buttons standing in for the readers) and the -DUSE_RFID=0 build (hex in wokwi/build/) let the controller be watched in the browser; that is a logic demonstration, not a hardware test.

7. Learnings

My first GPS implementation was worse than the baseline at every share because the EV green expired just as the EV arrived; that is why the controller became ETA-aware (EV_LEAD_MS, EV_CLEAR_MS, evProtectUntil) and why test_gps_eta_cut_is_timed_so_green_shows_before_arrival pins the behaviour. Earlier detection is not automatically better: GPS stops fewer EVs but holds greens longer, so the worst single non-EV wait rises to 144 s. Fairness has a price - a 10 s green clears only about 4 vehicles - and compensating a cut group with a longer next green is the obvious next improvement. In practice a city would apply this to electric buses and ambulances; the controller does not care what the tag is on.

KEY TAKEAWAY At the 5-10 % EV shares seen on Indian roads today the ETA-aware priority halves EV delay for a 4-9 % penalty on everyone else, with yellow, all-red and a 60 s starvation bound never violated; nothing ran on hardware.